

# Wellness Smart Shopping runsheet

Work top to bottom with the app closed to start. Each row says what to do and what you should see — if the two disagree, that's the bug. Tick the boxes as you go; note the row number for anything that fails.

## 01 Install from the disk image

The app is self-contained. Nothing needs to sit beside it.

☐ **Build the image, then open it**

Expect a window with the app, an **Applications** shortcut and a Read Me.

```
cd app && ./make_dmg.sh
open "dist/Wellness Smart Shopping.dmg"
```

☐ **Drag the app onto the Applications shortcut**

Copies in a second or two. Then eject the image.

☐ **Open it from Applications (or Launchpad)**

Launches with no Gatekeeper warning — it is ad-hoc signed.

☐ **Check Launchpad shows exactly one**

One **Wellness Smart Shopping**. Two means a stale copy survived.

## 02 Your daily targets

First run asks for the few details that size the shopping list to you.

☐ **Press Scan with Claude..., then Start Control Panel**

Progress ticks, then the panel opens in your browser.

☐ **The targets form opens by itself**

Only on a first run, when no profile is saved yet.

☐ **Enter height, weight, age, sex and activity**

The target updates **as you type**, no Save needed.

☐ **Switch activity from Sedentary to Very active**

Calories **and** protein both rise — training raises protein need, not just energy.

- ☐ **Switch goal between Maintenance, Cutting and Build Muscle**

Cutting lands below maintenance, Build Muscle above.

- ☐ **Clear the age field**

Falls back to a size-only estimate and says so, rather than breaking.

- ☐ **Press Save targets, then reload the page**

The card keeps your numbers; the form no longer opens on its own.

- ☐ **Press Edit on the targets card**

Reopens with your saved values in the fields.

### 03 **Make it yours**

Every greeting, heading and image is data, not code.

- ☐ **The extension notice appears near the top**

With **Get the extension** and **I already have it**.

- ☐ **Press I already have it, then reload**

Gone for good. It cannot truly detect the extension, so it asks rather than guesses.

- ☐ **Pick a different look**

Banner changes at once. 26 to choose from: six drawn, twenty photographs.

- ☐ **Drop your own image into the themes folder, reload**

Appears in the picker alongside the rest.

```
open ~/Library/Application\ Support/Wellness\ Smart\ Shopping/themes
```

- ☐ **Change the app name, greeting, tagline and deals heading**

Header updates live as you type.

- ☐ **Pick a shopping list message, or write your own**

The line above the deals changes to match.

- ☐ **Press Save, then reload**

All of it persists.

## 04 Scan with Claude

NEEDS THE EXTENSION

The heart of it: Claude reads deals off a store page you are signed in to.

### ☐ Press Scan with Claude in the panel

A store picker, each store marked scanned or not scanned yet.

### ☐ Pick ShopRite

Links to its coupon list **and** weekly ad — labelled differently, not twice the same.

### ☐ Press Copy on the instruction

Button confirms. The path in it is where your scan must be saved.

### ☐ Open the store, sign in, and give Claude that instruction

Claude scrolls the whole list and saves one offer per line.

### ☐ Leave the panel waiting

It notices within a few seconds — no button to press.

### ☐ Read the result

“N deals found. Import it now?” If nothing matched your staples it says so plainly instead of writing an empty file.

### ☐ Press Not now

Tells you exactly where the file is waiting. Declining loses nothing.

### ☐ Scan again and press Import deals now

Opens the app, reveals the file in Finder, path on your clipboard.

### ☐ Try it without a scan — try the sample instead

Ten matches, including a per-pound price converted to a per-package one.

```
python3 crawl.py --harvest shoprite=examples/sample-scan.txt --report
```

## 05 The deals are sound

A wrong match quietly corrupts your budget, so these are worth checking.

### ☐ Look at the reported basis for each deal

Per-pound converted to per-package, “2 for \$5” to a unit price, limits carried through.

☐ **Check what was *not* matched**

Junk is rejected: “Peanut Butter Cups” and “Cinnamon Toast Crunch” are not peanut butter and cinnamon.

☐ **Validate the XML against the catalog**

Prints `clean`. Unknown item IDs are silently dropped by the app, so this matters.

```
python3 -c "import sys;sys.path.insert(0,'.');\nfrom dealcrawler.xmlout import validate;\nprint(validate(open('out/shoprite-sales-2026-09-20.xml').read()) or 'clean')"
```

## 06 The desktop app

Planning and PDFs live here; scanning lives in the panel.

☐ **Read the button on the main window**

It says **Scan with Claude...** — never “Coupons (1)...”. The old coupon finder is deleted, not hidden.

☐ **Open its panel**

Four controls and no list of entry boxes: Start Control Panel, Import Sales XML, Clear, Close.

☐ **Choose Import Sales XML... and pick your file**

Offers the deals for confirmation, then applies them.

☐ **Close the panel, reopen it**

Your imported deals survive. Closing must never wipe them.

☐ **Click into the browser while the panel is open**

The window **stays put**; it does not vanish when the app loses focus.

☐ **Create the PDFs**

Shopping list, recipes and daily meals, priced against your imported deals.

☐ **Untick Apply deals to the list, create the list again, and compare it to the last one**

The **same items in the same order** — only the total differs. Deals are looked up after the list is settled, so they change what you pay, never what you buy. A different set of items here is a bug.

☐ **Tick it back on and look for a swap offer**

Any substitute stays inside one role — a protein for a protein, a grain for a grain — skips recipe-required items and favourites, and is yours to accept or decline.

☐ **Quit the app**

The control panel stops with it — the app started it, so the app stops it.

```
pgrep -fl panel.py    # expect nothing
```

☐ **Run its own self-test**

Ends `SELF_TEST_OK`, including `XML_IMPORT_OK`.

```
"/Applications/Wellness Smart Shopping.app/Contents/MacOS/SmartShoppingList" \  
--self-test /tmp/a.pdf /tmp/b.pdf /tmp/c.pdf
```

## 07 Where your files live

An installed app is read-only and signed, so nothing may be written inside it.

☐ **Look in Application Support**

```
config.json , branding.json , profile.json , harvest/ , out/ , themes/ .
```

```
open ~/Library/Application\ Support/Wellness\ Smart\ Shopping
```

☐ **Confirm nothing was written inside the app**

No output. A write in there would break the signature.

```
find "/Applications/Wellness Smart Shopping.app" \  
-name "*.pyc" -o -name "config.json" -o -name "__pycache__"
```

☐ **Re-verify the signature after a full session**

Silence means valid.

```
codesign --verify --deep --strict "/Applications/Wellness Smart Shopping.app"
```

☐ **Confirm it never phones home**

Passes. The crawler makes **no network requests at all**; deals come only from a scan you ran.

```
python3 test_crawler.py 2>&1 | tail -3    # 116 tests
```

## 08 Off again, cleanly

DO LAST

Anything you give away should come off without leaving a mess.

### ☐ Check first, change nothing

Reports every copy it can find and creates **nothing**.

```
python3 installer.py --check
```

### ☐ Uninstall

App, preferences, caches, WebKit and HTTPStorages all go. Your shopping data is asked about separately.

```
python3 installer.py --uninstall
```

### ☐ Check Launchpad and Spotlight

No ghost entry — removals deregister from LaunchServices, not just delete files.

**If a check fails**, note the row number and what you saw instead. The gap between “what to do” and “what you should see” is usually enough to find it.

---

Wellness Smart Shopping · free and open source, MIT.

Scanning needs Google Chrome or another Chromium browser, the Claude for Chrome extension and a Claude subscription. Everything else works without them.